

Ensemble early exit network on human activity recognition using wearable sensors

Jianglai Yu^a, Lei Zhang^a, Dongzhou Cheng^a, Can Bu^a, Liangdong Liu^a, Hao Wu^b, Aiguo Song^c

^a School of Electrical and Automation Engineering, Nanjing Normal University, 210023, Nanjing, China

^b School of Information Science and Engineering, Yunnan University, 650500, Kunming, China

^c School of Instrument Science and Engineering, Southeast University, 210096, Nanjing, China

ARTICLE INFO

Keywords:

Human activity recognition

Deep learning

Early-exit

ABSTRACT

Deep learning has recently achieved significant success in sensor-based human activity recognition (HAR). However, achieving higher levels of accuracy in these models often requires more computational resources. When such systems are deployed on edge devices with strict requirements for computing, memory, and communication, controlling energy consumption and inference latency becomes a challenging task. In the field of computer vision or natural language processing, it is recognized that not all inputs require the same amount of computation to produce reliable predictions. Adaptive inference, as a prominent approach for efficient deployment, has gained increasing attention. In particular, the ensemble early-exit mechanism has emerged as a promising direction for adjusting the computational depth of each input sample at runtime, providing better accuracy-cost trade-off for deep learning models. However, the “overthinking” problem has been rarely explored in the domain of HAR. In this paper, we for the first time leverage such ensemble early-exit mechanism to investigate the “overthinking” problem in the context of HAR. By cascading multiple exits in the early-exit network, we analyze the variations of activity prediction within the network, address the “overthinking” problem in early-exit activity recognition scenario, and propose strategies to mitigate it. Experimental results on multiple mainstream HAR benchmarks demonstrate that our approach allows for adjusting the computational load based on the difficulty levels of different sensor samples, leading to improved classification accuracy for various early-exit activity recognition tasks. A practical on-device latency analysis is provided. Our code will be released at: <https://github.com/Yuki0614/Ensemble-Early-Exit-Network-for-HAR>.

1. Introduction

1.1. Background

The past decade has witnessed rapid advancement of sensor technology according to several crucial performance indicators (i.e., higher accuracy, smaller size, and lower manufacturing price), which enables a large variety of motion sensors (e.g., accelerometers and gyroscopes) to be incorporated into various portable Internet-of-Things (IoT) and wearable devices like smartphones and smartwatches. Due to its prominent advantages, sensor-based human activity recognition (HAR) has gained wide popularity in ubiquitous computing community, which could help these smart sensing devices to better understand human behavior through analyzing raw sensor data collected from heterogeneous sensors attached to different positions of human body,

thus providing assistive services for improving their quality of life in a wide range of application scenarios including sports tracking, elderly care, smart homes, healthcare, and Human-Computer Interaction (HCI) [1,2]. Traditional machine learning (ML) algorithms [3] such as Random Forest [4], Naïve Bayes [5], and Support Vector Machine (SVM) [6] have always played an important role in activity recognition, which may be perceived as a multichannel time series classification problem. However, they are largely constrained by laborious and time-consuming manual feature engineering [3,7] that relies on specific domain knowledge or expert experience. Recently, various deep neural networks (DNNs), especially convolutional neural networks (CNNs) equipped with an automatic ability of feature extraction are extensively exploited to address above challenges, which have achieved state-of-the-art performance and triumphed over a multitude of HAR tasks [7, 8].

* Corresponding author.

E-mail addresses: 221802013@njnu.edu.cn (J. Yu), leizhang@njnu.edu.cn (L. Zhang), 211843002@njnu.edu.cn (D. Cheng), 211843007@njnu.edu.cn (C. Bu), 231812044@njnu.edu.cn (L. Liu), haowu@ynu.edu.cn (H. Wu), a.g.song@seu.edu.cn (A. Song).

<https://doi.org/10.1016/j.comnet.2025.111409>

Received 6 April 2024; Received in revised form 17 April 2025; Accepted 22 May 2025

Available online 6 June 2025

1389-1286/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

1.2. Current challenges

Despite tremendous success, most DNNs often yield high computational cost, which makes them hard to be afforded on mobile and wearable devices, hence suffering from inference delay. In fact, how to deploy deep learning models within given latency constraint will be of vital importance for many resource-sensitive HAR applications. Too long delay would have a negative impact on mobile user experience. The activity inference has to be done in real-time, that requires a short response time. Towards this goal, a series of lightweight DNNs manually designed have been introduced in [9–11], which have smaller computational costs, i.e., lower on-device latency. Most of the prevalent DNNs can only do inference in a static manner, i.e., both their model parameters and computational graph will always remain fixed or unchanged once trained is finished. However, at runtime these static DNNs are not easy to reconfigure or adapt across different devices, given the same response time constraint. For example, to the best of our knowledge, there are over 50 popular mobile phone brands (e.g., iPhone, Samsung, and Google) with different versions, which have variable computing capabilities. These heterogeneous devices would yield drastically different inference times for the same model [12]. Even for the same device, the runtime can also vary drastically (e.g., a mobile device suddenly turns into power-saving mode). Generally speaking, given the same response time constrain, high-performing devices can produce satisfactory accuracy by allocating more computations, while low-end devices have to accelerate inference by sacrificing accuracy. Dynamic neural network such as early-exit has recently emerged as a promising solution to reach a desired accuracy–efficiency trade-off, achieving outstanding success in computer vision (CV) and natural language processing (NLP) areas [13]. Despite notable advantages in terms of accuracy, efficiency, and adaptiveness, it still receives limited research attention in the context of sensor-based HAR. A natural question arises: given the same latency constraint, how to flexibly and adaptively trade-off between latency and accuracy at runtime for efficient activity recognition?

1.3. Research motivation

In principle, previous static DNNs often apply the same computations to all input samples during their decision-making process (i.e., the forward pass), where each activity sample has to pass through the whole network from layer to layer and then make a prediction while attaining the final layer, whether it is easy or difficult to recognize. That is to say, a DNN can gradually recognize activity features from coarse to fine through this layer-by-layer process [14]. Despite remarkable success of deep learning, the interior mechanisms of activity recognition based on DNNs now remain very opaque because they are only able to provide final prediction results, leading to a black-box property. It is necessary to interpret such deep decision behavior to explore how such activity prediction gradually evolves throughout all the layers, which still lacks an in-depth analysis. In this paper, we characterize a prevalent weakness of most existing DNNs termed as “overthinking” [15], when a DNN could produce correct predictions before its final layer. This is very likely to human overthinking, when one sometimes thinks too much. Such overthinking phenomenon has been long ignored in the context of HAR. Intuitively, we hypothesize that for an activity sample, its simpler feature representations are already enough to give a correct prediction at an earlier or shallower layer, rather than the final layer. We argue that further excessive computation after this current layer would cause unnecessary waste, which also slows down inference speed, just like people thinking too long. Such DNN overthinking over an activity sample would be computationally wasteful, and even potentially harmful or destructive while leading to confusion or misclassification. Motivated by such analogy, this paper aims to expose whether activity recognition using DNNs are susceptible to overthinking, from a new perspective of trading off between latency and accuracy.

1.4. Main contributions

In this paper, we provide an in-depth analysis to illuminate how activity prediction evolves from layer to layer by considering difficulty level of different activity samples, which makes the aforementioned black-box decision-making process more clear or interpretable. To this end, we propose an Ensemble Early-exit approach in ubiquitous HAR computing scenario, which can dynamically adapt the routing process of input activity samples according to their difficulty levels, leading to a better trade-off between latency and accuracy at a given resource constraint. In principle, our analysis is built on an idea called early-exit by attaching simple Internal Classifiers (ICs) called classification heads to several hidden layers of previous off-the-shelf DNNs, hence shortening the inference time while avoiding waste in further computation. Fig. 1 illustrates an overview of our proposed approach, where one can observe detailed routing process of “Lying” and “Going upstairs” activities in our analysis. Since the sensor signal waveform of the “lying” activity is simple, it can produce a highly confident prediction and exit early in the shallow layers of this DNN, while the “Going upstairs” activity is complex and easily confused with other activities, thus causing more computation.

Specifically, if a given IC is able to produce a sufficiently high confidence compared to a pre-defined threshold, the prediction will be returned, otherwise, this activity example will be passed to the subsequent classifiers exited later. On this basis, we explore how activity information available at different layers would contribute to the decision-making process of the entire DNN. However, during the forward pass, a correct internal prediction might evolve into a wrong one, i.e., misclassification. Moreover, in the evaluation of the current IC, such early-exit scheme only simply discards the prediction returned by early ICs, which potentially ignores valuable activity information (e.g., predicted confidence), while wasting previous computational efforts that have been already incurred. To avoid this issue, borrowing an idea of gradient boosting, we introduce an intelligent aggregation scheme of activity information that explicitly transmits the past information containing low-level features to the later ICs, rather than generating them from scratch. By reusing past information, the later ICs can combine the predictions from previous ICs and produce better predictions, thus effectively alleviating the problem of overthinking. From a perspective of information recycling use, our method has a fundamental difference from prior most early-exit strategies [15–17], which provide an opportunity for each IC to explicitly reuse predictions from all previous ICs, by additionally building an ensemble of shallow and deep ICs, without incurring extra computation burden. The main contributions of this paper are summarized as follows:

- First, most existing early-exit works often simply attach internal ICs to different stages of the entire decision-making process in DNNs, which neglect potential “overthinking” phenomenon, hence rendering model performance suboptimal. So far, such “overthinking” problem has received limited research attention in the context of HAR. In this paper, we for the first time provide a detailed performance analysis about the “overthinking” phenomenon in an ensemble early-exit activity recognition scenario, which cascades multiple weaker and shallower ICs into a stronger one, helping to resist against overthinking over simpler activity samples, without incurring additional inference cost.
- Second, we validate the effectiveness and efficiency of our approach on multiple public HAR benchmark datasets including UCI-HAR, PAMAP2, OPPORTUNITY, and UNIMIB-SHAR, as well as on a resource-limited mobile platform. To the best of our knowledge, this paper is the first to show that such Ensemble Early-exit strategy avoids the harmful or destructive effect of overthinking, which could effectively cut computations wasted by overthinking while preserving accuracy in a dynamic activity inference setting. Realistic on-device inference latency is given.

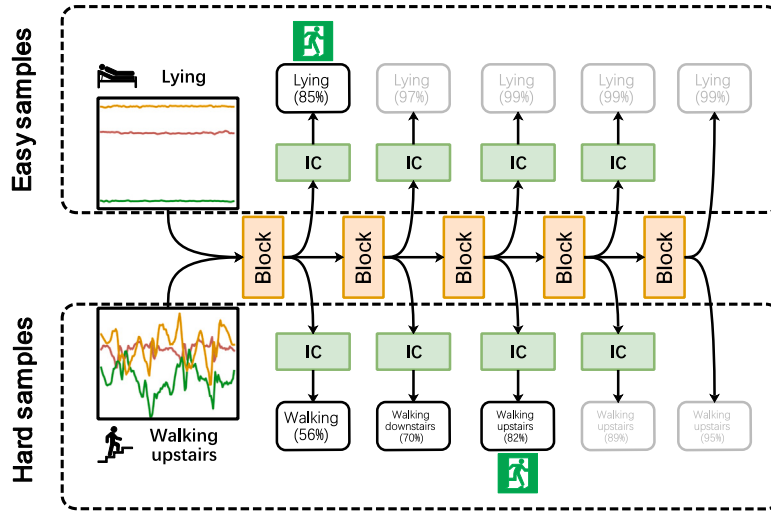


Fig. 1. Comparison of routing of easy and hard samples in Ensemble Early-exit Networks.

- Finally, based on a confidence-based early-exit procedure, a quantitative analysis about confusion or misclassifications caused by overthinking is provided to show that existing DNNs would overthink on the majority of input activity samples, which cause wasted computation and slow inference, indicating that such simple activity samples passing the full network depth are unnecessary. We also visualize the overthinking phenomenon from layer to layer by diagnosing the misclassifications caused by various activity samples, which provides a new perspective of reasoning for interior decision-making interpretability.

The remaining subsections of this paper are the following. Section 2 provides a survey of recent relevant literatures on deep learning and human activity recognition (HAR). Section 3 introduces an overview of the proposed ensemble early-exit framework. In Section 4, we conduct extensive experiments on several publicly available HAR datasets. Section 5 quantitatively studies the problem of overthinking in HAR scenario. Section 7 concludes this paper.

2. Related work

DNN-based HAR. During recent years, numerous DNNs, especially CNNs have made major breakthroughs in many HAR tasks due to their superior performance in automatically extracting features and identifying hidden or unknown activity patterns from raw time series sensor data [18,19]. Zeng et al. [20] at an earliest time develop a CNN-based feature extraction approach to capture the scale invariance and local dependency characteristics from raw sensor time series for activity recognition, significantly outperforming traditional ML algorithms that use manual features. Ignatov et al. [21] propose a novel CNN architecture that combines automatic feature extraction by CNN with shallow handcrafted features, so as to better preserve the global form of sensor signals for real-time activity recognition. Yang et al. [22] introduce a powerful CNN architecture that first applies convolution filters along temporal dimension for each sensor modality and then unifies all feature maps from different sensor modalities as an input fed into the final classifier. Ordonez et al. [23] propose a hybrid network architecture named DeepConvLSTM that replaces the fully connected layer of CNN with Long Short Term Memory (LSTM) unit to capture long-range temporal dependencies for activity recognition. Luo et al. [24] design a binarized CNN architecture for real-time HAR that uses dilated convolution to enlarge the receptive field and improve its potential for handling time series data, which could effectively reduce both activity inference latency and memory footprint in resource-constrained mobile devices. Kim et al. [25] propose an interpretable and accurate CNN

model capable of highlighting important sensor signals for activity recognition. Ma et al. [26] introduce a new attention-based DNN named AttnSense to address multimodal HAR problem, which combines an attention module with a CNN and a Gated Recurrent Units (GRU) network, showing prominent advantages in improving the comprehensibility. Though these DNNs have achieved promising results, they potentially ignore the difficulty levels of different activity samples. These previous literatures mainly focus on building more powerful and efficient DNNs for activity recognition, but less research attention has been devoted to early-exit activity recognition. Recently, Lattanzi [27] have at an earliest time systematically explored the use of early-exit mechanism in the context of HAR. Rashid et al. [28] have proposed a novel adaptive model named AHAR, which employs an output block predictor to select a portion of the backbone architecture. Different from ours, they mainly focus on the energy-efficient and memory-efficient HAR. Although Qendro et al. [29] have proposed an early-exit ensemble problem, they mainly attempt to handle the medical imaging classification tasks from a perspective of model calibration. Similarly, Sun et al. [30] have also explored such early-exit ensemble from a perspective of model calibration in the natural language processing (NLP) community. Zhou et al.'s [31] and Wolczyk et al. [32]'s early-exit works are the most similar to ours. The former seeks to improve the accuracy and robustness of a pretrained language model (PLM) by preventing it from overthinking, while the latter attempts to alleviate the overthinking phenomenon in computer vision tasks. To the best of our knowledge, different from these existing early-exit works, this paper is the first to delve into such early-exit activity recognition challenge from a perspective of overthinking. In particular, the interpretability of these previous DNNs still remains very opaque in the context of sensor-based HAR. Our work attempts to interpreting how the difficulty levels of different activity samples influence the underlying decision behaviors of DNNs in an ensemble early-exit activity recognition scenario.

Deep supervision and internal classifiers. Deeply supervised networks [33] and early-exit schemes [16] have been recently proposed to train internal classifiers for various computer vision tasks, where samples could be conditionally early-exited by adding internal classifiers to the network, i.e., they do not need to pass through the full network for reducing inference times. For example, Szegedy et al. [34] firstly propose to train ICs so as to regularize the network for improving overall accuracy. In case of dynamic computational budgets, Huang et al. [35] design an adaptive network architecture called MSDNet, which aims to produce any time predictions for image classification. Teerapittayanon et al. [16] introduce a new model named BranchyNets to augment DNNs for improving inference efficiency, which attaches the ICs to several convolutional layers and then jointly optimizes all

weighted loss from different exit points. Kaya et al. [15] present a simple yet effective Shallow-Deep Networks to select several early-exit points by comparing the predicted confidence (i.e., maximum of the softmax probability) with a pre-defined threshold, where the response of each IC is independent of the responses of the previous ICs, i.e., it does not reuse the past information. On the other hand, the concept of deep supervision has been for the first time proposed by Lee et al. [33] in 2015. The past decade has witnessed its outstanding success in a large variety of computer vision applications such as object detection [36], image super-resolution [37], and semantic segmentation [38]. Recently, Li et al. have also provided a comprehensive and in-depth survey paper about deep supervision in computer vision community [39]. However, the idea of deep-supervision is rarely to be seen in early-exit HAR scenario from a perspective of overthinking. Hence, in this paper, we focus on adaptively accelerating activity inference from a perspective of alleviating further overthinking over simple sensor samples. Despite being very effective in various areas such as CV and NLP, the ensemble early-exit mechanism has remains less explored in the context of HAR. Without any specific assumption about the underlying network structure, we apply such ensemble early-exit mechanism to cascade multiple internal classifiers, which focus on remedying or mitigating the wasteful and destructive effect of overthinking problem for early-exit activity recognition, hence achieving two goals previously perceived to be contradictory: adaptively accelerating activity inference while preserving satisfactory classification performance.

3. Ensemble early-exiting network

Most existing works assume the same computation budget for all activity samples no matter how difficult they are [27]. Intuitively, difficult activity examples usually need more computational budgets to process. Instead, consuming the same computational budget for easy activity examples might be wasteful and even harmful. Motivated by the observation, the main goal of this paper is to accelerate activity inference of DNNs by mitigating overthinking problems and reducing redundant computations. To this end, using an idea of conditional computation, we decide the routing of input activity samples according to their difficulty levels processed by a DNN. By dynamically adapting the route of computation, we can control the computational budget of each particular activity sample to achieve a better trade-off between latency and accuracy.

Hence, we specifically focus on how to modify off-the-shelf DNNs in a deeply-supervised setting. The standard DNN typically contains a sequence of internal layers, which end with a final classifier, often a fully connected layer. To perform dynamic routing over input activity samples, we insert a set of internal classifiers (i.e., ICs) after several internal layers. During the forward pass process, every IC can output intermediate predictions based on the features extracted from the previous internal layer. If the obtained confidence can meet the early-exit conditions, the prediction will be returned in advance without performing further computation through the subsequent layers, thereby saving computational resources. Otherwise, assuming that current IC is insufficient to produce confident prediction, the activity sample has to be passed to the subsequent layers for further computations. Applying ICs to output an earlier response could reduce overall inference time. However, in such a setting, every IC only makes its own decision independently, which potentially ignores the past information generated by previous ICs. As aforementioned, because previous ICs might sometimes output correct predictions for activity examples that are misclassified by later ICs, simply discarding the past information would cause wasteful computations and performance drops. To avoid this issue, one has to collect the past information from previous ICs to help the decision of current IC. For this purpose of reusing information, we ensemble earlier ICs and later ICs (as well as the final classifier) into a single one by cascading connections, thereby yielding improved model performance. This design could mitigate the limitation of a single IC via ensemble learning while preventing wasteful computations from later ICs and, in turn, accelerate activity inference. Below we will detail this Ensemble Early-exit scheme, as shown in Fig. 2.

3.1. Model overview

Our model consists of three main components, i.e., the internal classifier (IC), the backbone network and the ensemble layer. Without loss of generality, the backbone network is a standard DNN structure comprised of N convolutional layers and a final classifier. At a given convolutional layer F_m , we add the internal classifier denoted as IC_m ($1 < m < N$). Generally speaking, the index m is smaller than the overall number of internal layers within the DNN, since in some cases one does not attach ICs to each layer. Each IC is composed of a fully connected layer. For the specific m th internal classifier (IC_m), the fully connected layer generates internal predictions p_m . The ensemble layer followed the IC and integrates the inferred information from the current IC and all previous ICs via cascading connections to produce the final prediction q_m at the current IC. The addition operation of the ensemble layer allows the already inferred information to be transmitted between consecutive ICs, and later ICs can directly obtain past information that has already been calculated at almost no computational cost. If earlier ICs have no enough predictive power to recognize activities correctly based on low-level features, ensembling more subsequent ICs can gradually improve model performance as the number of ICs grows, hence demonstrating greater improvements in deeper layer of the DNN. The details will be introduced in the following subsections.

3.2. Internal Classifier(IC)

As illustrated in Fig. 2, each IC consists a fully connected layer. To make our modification (i.e., attaching ICs) to be generic, here we opt for directly employing a fully connected layer rather than more complex structures like a multilayer perceptron or a convolutional layer, which allows us to closely observe how activity recognition evolves during the forward pass while monitoring the presence of overthinking phenomenon, but has minimal interference from these attached ICs. Furthermore, the fully connected layer may also be perceived to be the feature reduction for scalability, due to big output size from an internal layer. For convenience of notation, given an input x , let $g_m(\cdot)$ be the fully connected layer of IC_m , and $F_1(\cdot), F_2(\cdot), \dots, F_m(\cdot)$ be the internal layers that have been previously passed through before IC_m in the backbone network. The internal IC prediction $p_m(x)$ can be expressed as follows:

$$p_m(x) = \text{softmax} \left(g_m \left(F_m \left(F_{m-1} \dots \left(F_1(x) \right) \right) \right) \right). \quad (1)$$

3.3. Ensembling ICs

In machine learning community, ensemble learning has become a mainstream performance-boosting technique, which can reliably improve model performance while increasing uncertainty estimation and robustness. Despite higher accuracy, its obvious drawback lies in wastefulness, which requires training multiple models for the same example. To avoid this issue, we aim to leverage such idea of ensemble learning by combining pre-computed predictions from previous ICs in our setting, incurring near-zero additional computational overhead. That is to say, to realize such near-zero waste, we implement such ensembles by combining responses from previous ICs to yield the final response at the m th classifier. Because different ICs might have a variable predictive power (e.g., later ICs typically outperform earlier ones) and there is a potential correlation between their consecutive predictions, it is necessary to employ a weighted approach to integrate the individual internal predictions. Following previous literature [32,40,41], a weighted geometric mean ensemble strategy is utilized to find a proper combination of pre-computed responses, so as to maximize the expected result. As defined above, for an input x , assuming that $p_1, p_2, \dots, p_m$ are the internal prediction of the fully connected layers within overall

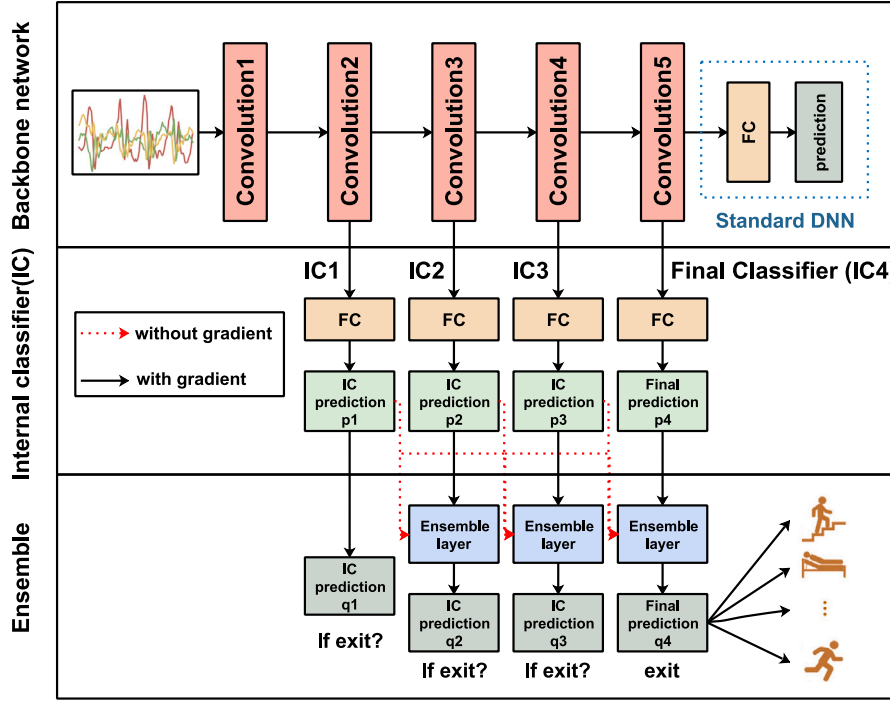


Fig. 2. Detailed scheme of proposed Ensemble Early-exit Network.

m ICs, we can formulate the probability of the i th class in the m th ensemble layer as:

$$q_m^i(x) = \frac{1}{Z_m} b_m^i \prod_{j \leq m} (p_j^i(x))^{\omega_m^j}, \quad (2)$$

in which both $\omega_m^j > 0$ and $b_m^i > 0$, (e.g., $j = 1, \dots, m$) denote the trainable weights and bias terms respectively, while Z_m can be seen as a normalization factor that satisfies $\sum_i q_m^i(x) = 1$. Because the weight and bias parameters are independently trained for each subset $\{IC_j : j \leq m\}$, the m indices in both ω_m and b_m are needed. Here the weight parameter ω_m^j can be perceived as our prior belief in the predictions of IC_j , where a higher weight indicates lower confidence in these predictions. Moreover, the bias parameter b_m^i denotes the prior probability of the i th class for IC_m . Though it is also feasible to tune these parameters manually, our experiment results validate that directly applying standard cross-entropy loss to optimize them on training set performs the best.

It is worth noting that directly computing the product in Eq. (2) would cause numerical instabilities, if the probabilities approach zero. To circumvent this issue, Eq. (2) may be transformed into:

$$b_m^i \prod_{j \leq m} (p_j^i(x))^{\omega_m^j} = b_m^i \exp\left(\sum_{j \leq m} \omega_m^j \ln(p_j^i(x))\right), \quad (3)$$

where the log-probabilities $\ln(p_j^i(x))$ can be calculated through the log softmax function on the logits g_{ϕ_m} of IC, that will be more numerically stable. In above formulations, we highlight that such a geometric ensemble setting is preferable to additive one. If a single IC has a low confidence for a particular activity class, it will substantially decrease the likelihood of this given activity class in the entire ensemble. To make the confidence of the corresponding activity class to be as high as possible, one has all ICs to be highly confident in that class, suggesting that an incorrect yet confident IC response would have a lower probability of prematurely terminating calculations in such geometric ensemble setting. Thus, here the geometric ensemble setting is used by default. In such a setting, ensembling can help to implicitly improve the performance of later ICs by combining the past information from previous ICs. It is worth mentioning that, during forward propagation,

the predictions of the previous ICs, which are passed to the ensemble layer that corresponds to the current IC, do not have gradients. This is because, in theory, early predictions should not be influenced by subsequent layers.

3.4. Model training

In principle, training standard DNN primarily aims at improving the final prediction. To observe an overthinking phenomenon over activity inference, we do not freeze the backbone DNN weights. Instead, we choose to train the backbone DNN and the ICs together in this paper. Intuitively, the IC-only training can keep the DNN as intact as possible and is also fast, since the gradient backpropagation only updates the weights in these ICs. However, it has a major drawback: different ICs have a varying predictive power, as a result, leading to different training epochs required, e.g., some ICs have not converged sufficiently while others have overfitted due to excessive training. In our setting, independently-training would result in weaker ICs when adding these ICs on top of these internal layers and training them in a separate manner. We highlight that our main goal is to modify the off-the-shelf DNN for mitigating the overthinking problem. Towards this goal, instead of Uniform Weights Loss (UWL) [15], we employ an Ascending Weights Loss (AWL) [31] to jointly train the original DNN plus attached IC weights from scratch, by combining the intermediate losses generated by all the ICs into a total loss. Without loss of generality, let $L_1, L_2, \dots, L_m$ be the loss function of all attached ICs, and L_{m+1} be the loss function of the final classifier. We train the DNN by minimizing the following total loss function with an ascending weight average:

$$L_{All} = \frac{\sum_{i=1}^{m+1} i \cdot L_i}{\sum_{i=1}^{m+1} i}. \quad (4)$$

In this formula, such ascending weight average can correspond to the relative inference cost of each IC, and the later ICs will receive higher weights, indicating that it will receive more sufficient or comprehensive training as well, which is consistent with our intuition. Since the gradients often tend to vanish at deeper layers, the higher weights can successfully magnify the gradients when implementing back-propagation, hence leading to higher overall accuracy. In such a

Algorithm 1 Ensemble Early-exit Network

Input: Ensemble Early-exit Network, Sensor data x
Output: Predicted probability distributions $Output_i$

```

1: Setting threshold  $\tau$ 
2: for  $i, F_i, g_i$  in zip([1, 2, ..., n], Backbone model, Internal Classifiers)
   do
3:    $x = f_i(x)$ 
4:    $Output_i = g_i(x)$ 
5:   if  $\max(\text{softmax}(Output_i)) > \text{threshold } \tau$  then
6:     return  $Output_i$ 
7:   end if
8: end for
9: return  $Output_n$ 

```

way, each possible early-exit branch can be fully covered in the course of training.

3.5. Conditional inference

Once training is finished, the next step is to solve how to deploy the constructed system for activity inference, i.e., it needs to adaptively decide the shortest inference path for a particular activity example in the real-world. To accomplish this, we use the probability of the most confident class to define a confidence score, and then compare it with a pre-defined threshold τ [13,16,35]. If the m th IC is sufficiently confident about its prediction according to the following condition:

$$\max_i q_m^i > \tau, 0 < \tau < 1, \quad (5)$$

where i denotes the index of activity class, then the inference process will halt and output the response from current IC as the final prediction. As aforementioned, τ is a pre-defined confidence threshold, which can be empirically determined based on the available computational budget [35]. Otherwise, the activity sample will be passed to the next IC for further calculation. This procedure will be repeated until this condition is satisfied or the final classifier is already attained. In fact, all intermediate predicted probability scores strictly follow a monotonically increasing trend. It is worth noting that the threshold of final classifier is always to be zero, which ensure that the final classifier can reach the threshold. In any case, all activity sample would be forced to exit when they arrive at the final classifier. The specific process is displayed in Algorithm 1.

It is worth noting that though setting a lower threshold can yield a significant speed-up for activity inference, it would also lead to a substantial performance drop. Instead, when τ is set to be always larger than one, an original DNN inference f_θ will be recovered, because none of these ICs can be confident enough to provide an earlier answer. Therefore, the threshold values τ need to be carefully tuned to achieve a better trade-off between latency and accuracy according to practical requirements. Following earlier works [13], we leave a held-out set to test a range of possible values of τ in a budgeted batch classification [35] setting, which will be describe in Section 4.2.2.

4. Experiments

The whole section is divided into three subsections: The first subsection details the four publicly available HAR benchmarks we have used, namely UCI-HAR, PAMAP2, OPPORTUNITY and UNIMIB-SHAR. The second subsection introduces the standard data preprocessing pipeline. The third subsection provides the implementational details and network architectures of our baseline backbones. In the fourth subsection, the effectiveness and efficiency of our proposed approach are evaluated in two settings.

4.1. Experimental setting

4.1.1. Datasets

The UCI-HAR Dataset: The research team from Irvine, University of California has created the dataset to test various machine learning algorithms for activity recognition [6]. To aggregate this dataset, a group of total 30 volunteers aged from 19 to 48 years are enlisted to perform 6 types of common daily activities, namely, “Walking”, “Climbing Upstairs”, “Descending Stairs”, “Sitting”, “Standing”, and “Lying” - while wearing a Samsung Galaxy S II smartphone around their waists. The triaxial accelerometer and gyroscope embedded in the smartphone are used to record activity signals at a constant sampling frequency of 50 Hz. We follow the same setting as [42]: 70% of all these volunteers is selected to generate the training set and 20% the test set (10% is held out for validation).

The PAMAP2 Dataset: The Physical Activity Monitoring for Aging People 2 (PAMAP2) dataset can be utilized for activity recognition, which is collected from three inertial measurement units (IMUs) attached to each dominant’s hand, chest, and side ankle respectively [43]. During data collection process, 9 participants are asked to perform 18 different kinds of daily activities such as “Walking”, “Lying”, “cycling”, “Playing Soccer”, “Watching TV”, “Car Driving”, “Computer work”, “Rope jumping”, etc. The sampling frequency is constantly set to 100 Hz, which is further subsampled into 33.3 Hz. A heart rate is used to estimate activity intensity at a sampling frequency of round 9 Hz. We follow the same training-validation-test splitting protocol [44]: we choose the first and second runs from participant 6 for test, and the first and second runs from participant 5 for validation. The rest of the data is leaved as training set.

The OPPORTUNITY Dataset: This dataset is built by Daniel et al. from the University of Sussex in a sensor-rich environment equipped with 15 wired and wireless sensor systems, comprising total 72 sensors of 10 modalities [45]. We employ the subset of this dataset from the “OPPORTUNITY Activity Recognition Challenge” for our evaluation. In a breakfast-preparing scenario, four subjects are instructed to perform 17 distinct types of activities such as “Preparing Coffee”, “Drinking Coffee”, “Preparing Sandwich”, “Eating Sandwich”, “Cleaning Table”, which are sampled at a constant frequency of 30 Hz. For a fair comparison, we replicate previous splitting protocol in [46]: the fourth and fifth runs performed by subjects 2 and 3 are held out for test, while run 2 from subject 1 is leaved for validation. The remaining data is utilized for training.

The UNIMIB-SHAR Dataset: The dataset is comprised of total 11,771 samples performed by 30 volunteers aged between 18 and 60 years old [47]. A Samsung Galaxy Nexus I9250 smartphone integrating a Bosh BMA220 3D accelerometer is used to record sensor signals at a frequency of 50 Hz. All the samples belong to 17 fine-grained classes and can be further categorized into two coarse-grained classes: 9 types of activities of daily living (e.g., “Running”, “Jumping”, “Sitting Down”) and 8 types of falls (e.g., “Falling Rightward”, “Falling Leftward”, “Syncope”, “Generic Falling Backward”), which are primarily intended for human pose estimation and fall detection. Based on the setting of original dataset [47], we present the leave-one-subject-out cross validation.

4.1.2. Data preprocessing pipeline

Data preprocessing including noise filtering, data normalization, segmentation, etc. always plays a crucial role in activity recognition scenario. In particular, time series sensor signals need to be first segmented by sliding window into a series of fixed-length windows for further analysis. An overlapping rate is allowed between adjoining windows to preserve continuous nature of sensor signals. However, so far, there has been no clear consensus or well-established solution regarding the optimal window size for various HAR tasks, resulting in potential limitation and ambiguity. From an empirical perspective, the window length has an essential effect on the trade-offs between

Table 1
Summary of setup for datasets.

Datasets	UCI-HAR	PAMAP2	UNIMIB-SHAR	OPPORTUNITY
Categories	6	12	17	18
Sample (Hz)	50	33.3	50	30
Window	128	171	151	30
Overlap (%)	50	78	50	50
Epoch	200	200	200	200
Batchsize	128	128	128	128
Learning rate	1e-4	1e-4	1e-4	1e-4
Noise filtering/Standardization ✓				

inference time and accuracy. Generally speaking, reducing the window length leads to a faster activity inference (i.e., decreased computational budgets and energy consumption. Instead, increasing window length is often suitable for recognizing complex activities that last a relatively long time period, hence slowing down activity inference. To facilitate fair comparisons, we still adopt the same sliding window protocols from previously successful cases listed in early literatures [21,44,46]. Table 1 details the data preprocessing pipeline including the sampling rate, window length, and overlapping rate, etc.

4.1.3. Implementational details

To evaluate the proposed approach in supervised setting, we select four standard activity classification benchmarks including UCI-HAR, PAMAP2, OPPORTUNITY, UNIMIB-SHAR, and the previously popular CNN architectures [15,44], as our backbone network. Most popular architectures besides the convolutional ones we are using, may be divided into blocks (e.g., residual blocks in residual DNNs). Since each block would potentially change the dimensionality of the output features, a natural choice is to attach an IC after every block, which can considerably simplify actual implementation and also facilitate the extension of our approach for any future architectures. Without loss of generality, the backbone networks are comprised of five convolutional layers followed by a batch normalization layer and a non-linear activation function ReLU, where the ensembled ICs are inserted after each convolutional layer. Table 2 illustrates the details about backbone network architectures including the convolutional kernel size, stride, and channel number, which are in charge of controlling the size of output feature map at each layer. On this basis, we examine how much inference cost we could save while preserving satisfactory performance by reusing past information of ICs. Towards this goal, we measure the average number of FLOPs (i.e., floating-point operations) required for an activity sample during the forward pass phase, which can be perceived as a hardware-agnostic metric (i.e., indirect metric) for computational cost. We additionally check how such ensembled early-exit performs at avoiding waste by measuring practical inference time (i.e., direct metric) in a real hardware in the later section. As introduced in Section 3.4, a weighted loss function is employed to train the original backbone networks jointly with these ensembled ICs from scratch. In order to ensure convergence, we implement an SGD optimizer on the training dataset through 200 epochs with an initial learning rate (e.g., 10e-4), which is lowered by a factor of 10 after every 50 epochs. The batch size is constantly set to 128. We conducted ablation experiments on the UCI-HAR dataset to verify the effectiveness of various loss functions. We compared three loss functions, including Ascending Weights Loss (AWL) [31], Uniform Weights Loss (UWL) [15] and Descending Weights Loss (DWL) which calculated by $L_{All} = \frac{\sum_{i=1}^{m+1} (m+2-i) \cdot L_i}{\sum_{i=1}^{m+1} i}$ under the same settings as Eq. (4),

and the results are shown in Fig. 3. By comparing the accuracy of the final classifiers, we found that AWL was significantly more effective than the remaining methods, with AWL typically converging faster and achieving a 2% and 1% accuracy gain over unweighted joint loss and DWL, respectively, at the end of training. Table 1 details the related hyperparameters during training process. All programs are run using

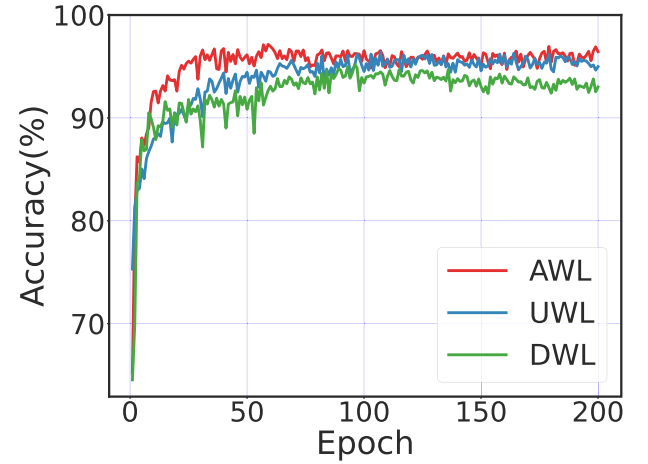


Fig. 3. Accuracy in different training ways on UCI-HAR dataset.

current mainstream deep learning library PyTorch. In order to conduct a more comprehensive evaluation of the proposed method, two main performance metrics were employed, namely accuracy and F-measure, with their mathematical expressions defined as follows:

$$Precision = \frac{TP}{TP + FP}, \quad (6)$$

$$Recall = \frac{TP}{TP + FN}, \quad (7)$$

$$F - measure = 2 \times \frac{Precision \times Recall}{Precision + Recall}, \quad (8)$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (9)$$

where TP, FN, FP, and TN represent true positives, false negatives, false positives, and true negatives respectively.

4.2. Experiments results and performance comparison

4.2.1. The performance in setting inference costs

Table 3 displays the accuracy results for the early-exit network (Independent ICs), ensemble early-exit network and baseline CNN in four HAR datasets. Our model's accuracy is assessed under three budget settings, which are constrained to stay around 10%, 30%, and 70% of the original CNN's inference cost. Furthermore, we present the highest accuracy attained by early-exits without any predetermined budget limitations. The threshold value, τ , can restrict the maximum average inference budget (measured in floating point operations).

The results indicate that except 10% budget constraint, our method can consistently achieve an improvement of round 1% to 1.5% in accuracy compared to the baseline early-exit network. In fact, the 10% budget constraint suggests that all activity samples would be forced to exit earlier from the initial classifier. At an initial stage, our ensemble mechanism has not yet taken into effect. This improvement is attributed to the ensemble layer that integrate all internal outputs. As a result, it can be observed that as the samples progress further into the network, the integration results in a higher improvement compared to the baseline early-exit network. In the absence of integration, the early-exit network, when compared to CNN, exhibits a slight decrease in accuracy by approximately 1%. However, the ensemble early-exit network compensates for the accuracy loss of the early-exit network by aggregating predictions from multiple exits. As a result, the final prediction accuracy achieves improvements ranging from 0.2% to 1.1% compared to CNN on the four datasets.

In the UCI-HAR dataset, the ensemble early-exit network achieves accuracy improvements of 1.5% and 1% compared to the early-exit network (Independent ICs) and CNN, respectively, under the MAX budget

Table 2

Architecture of CNN with ICs ('K' means kernel size, 'S' means stride, 'C' means output channel).

	UCI-HAR	PAMAP2	UNIMIB-SHAR	OPPORTUNITY
Layer1	K(6,1),S(3,1),C16	K(6,1),S(3,1),C16	K(6,1),S(3,1),C32	K(3,1),S(2,1),C16
Layer2(+IC)	K(3,1),S(1,1),C16	K(3,1),S(1,1),C16	K(3,1),S(1,1),C32	K(3,1),S(1,1),C16
Layer3(+IC)	K(6,1),S(3,1),C64	K(6,1),S(3,1),C64	K(6,1),S(3,1),C128	K(3,1),S(2,1),C64
Layer4(+IC)	K(3,1),S(1,1),C64	K(3,1),S(1,1),C64	K(3,1),S(1,1),C128	K(3,1),S(1,1),C64
Layer5	K(6,1),S(3,1),C128	K(6,1),S(3,1),C128	K(6,1),S(3,1),C256	K(3,1),S(2,1),C128
Fully connected layer				

Table 3

Accuracy (%) & F1 score (%) of models on four benchmark datasets.

Datasets	Classifier	1(<10%)	2(<30%)	3(<70%)	4(MAX)	Others' results
UCI-HAR (4.4M & 96.2 & 95.8 ^a)	FLOPs(M)	0.4	1.2	2.7	4.5	30.8M & 96.4 [21]
	Independent ICs	90.4 & 89.9 ^a	93.5 & 93.0 ^a	95.1 & 94.7 ^a	95.7 & 95.4 ^a	38.2M & 95.4 [48]
	Ensemble(Ours)	90.8 & 90.3^a	94.4 & 94.1^a	96.3 & 95.8^a	97.2 & 97.0^a	17.5M & 96.5 [49]
PAMAP2 (51.4M & 90.8 & 90.5 ^a)	FLOPs(M)	4.8	14.4	32.1	54.2	37.1M & 90.0 [44]
	Independent ICs	86.4 & 85.8 ^a	89.6 & 89.2 ^a	89.8 & 89.5 ^a	89.9 & 89.5 ^a	39.6M & 90.3 [50]
	Ensemble(Ours)	86.9 & 86.5^a	90.5 & 90.3^a	90.8 & 90.6^a	91.0 & 90.6^a	44.8M & 89.3 [26]
UNIMIB-SHAR (7.1M & 75.5 & 75.1 ^a)	FLOPs(M)	0.6	1.9	4.4	7.4	5.8M & 74.3 ^a [51]
	Independent ICs	69.9 & 69.4 ^a	71.9 & 71.4 ^a	73.9 & 73.6 ^a	75.4 & 75.1 ^a	54.2M & 76.4 ^a [52]
	Ensemble(Ours)	70.3 & 70.0^a	72.6 & 72.4^a	74.8 & 74.5^a	76.6 & 76.4^a	21.0M & 74.7 [47]
OPPORTUNITY (28.3M & 81.5 & 81.3 ^a)	FLOPs(M)	2.1	6.1	18.4	30.7	105.0M & 76.8 [20]
	Independent ICs	78.5 & 78.1 ^a	79.0 & 78.8 ^a	80.5 & 80.1 ^a	80.7 & 80.4 ^a	8.1G & 77.1 ^a [53]
	Ensemble(Ours)	78.7 & 78.2^a	79.6 & 79.3^a	81.4 & 81.2^a	81.7 & 81.5^a	276.8M & 78.9 [23]

^a Means F1 score.

setting. Even under the 70% budget setting, our method still outperforms CNN. At the lowest 10% budget, the ensemble early-exit network achieves an accuracy of 94.4% compared to CNN. In the PAMAP2 dataset, under the highest budget condition, the ensemble early-exit network achieves improvements of 1.1% and 0.2% in accuracy compared to the early-exit network(Independent ICs) and CNN, respectively. Under the 70% budget condition, it performs on par with CNN. At the lowest 10% budget, the ensemble early-exit network achieves an accuracy of 95.7% compared to CNN. In the UNIMIB-SHAR dataset, under the maximum budget condition, our network's F1 score improves by 1.3% compared to the other two networks. Here it is worth noting that there exists severe class-imbalance problem in this dataset, where one class (i.e., fall action) of observations significantly underweights the other. In such cases, accuracy alone may be misleading. The F1 score would provide a more robust evaluation metric, especially while the minority class is of particular interest. In the OPPOTUNITY dataset, our proposed network outperforms the other two networks under the maximum budget condition. At a 10% budget, our network achieves an accuracy of 96.6% compared to CNN.

Additionally, we conducted a comparison between ensemble early-exit network and several SOTA algorithms for activity recognition. It is worth noting that the performance of the compared ensemble early-exit networks is based on the MAX budget. The results are summarized in Table 3. It can be observed that our ensemble early-exit network achieves an accuracy improvement of 0.8% on the UCI-HAR dataset compared to Ignatov et al.'s [21] approach, which combines CNN with handcrafted statistical features for activity feature extraction. Furthermore, our method outperforms Dong et al.'s [48] and Yang's [49] results by 1.8% and 0.7% in accuracy respectively. In the case of the PAMAP2 dataset, compared to Ma et al. [26] and Zeng et al. [44] who utilize attention mechanisms, our ensemble early-exit network demonstrates significant advantages with accuracy improvements of 1.7% and 1.0% respectively. Compared to Chen et al.'s multi-agent spatial-temporal attention model, our method achieves a 0.7% accuracy improvement. For the UNIMIB-SHAR dataset, it is observed that our ensemble early-exit network outperforms Li et al.'s [51] method fusing soft-assignment and CNN-LSTM architecture, achieving a 2.1%

Table 4

Integration with DeepConvLSTM backbone on OPPOTUNITY dataset (Acc).

Backbone	DeepConvLSTM			
Classifier	1	2	3	4
FLOPs(M)	8.3	18.7	36.2	50.4
Independent ICs	79.4	80.4	80.9	81.3
Ensemble ICs	80.0	81.5	82.1	82.6

improvement in F1 score. Long et al. [52] proposed a novel asymmetric residual network (ARN) that combines long-time windows and short-time windows. Remarkably, our method achieves the same performance as ARN in terms of F1 score. For the OPPOTUNITY dataset, the proposed ensemble early-exit network demonstrates a significant increase in accuracy by 4.9% compared to the convolutional architecture presented by Zeng et al. [20]. Furthermore, when compared to the DeepConvLSTM model proposed by Ordóñez et al. [23], there is an improvement in accuracy by 2.9%. In addition to accuracy, we also compare our proposed approach with static CNN with its each independent ICs with the same computation, as well as numerous existing popular baseline approaches from the literatures [20,21,23,26,44,47–53] in terms of FLOPs. As shown in Table 3, it can be observed that our proposed approach can significantly achieve competitive performance compared to these state-of-the-art activity recognition models, with a smaller amount of computation (i.e., FLOPs).

To highlight the benefits of our proposed approach to a greater extent, we perform an additional generalization analysis by embedding such ensemble early-exit mechanism to recent state-of-the-art DeepConvLSTM backbone [23]. Without loss of generality, we evaluate the extensibility of our method on the OPPOTUNITY dataset, by comparing it with the early-exit DeepConvLSTM baseline at different individual exits points. As shown in Table 4, it can be seen that our approach considerably beats the comparing early-exit DeepConvLSTM baseline by 0.6%, 1.1%, 1.2%, and 1.3% at different exit points, with the same amount of computation. In summary, our ensemble early-exit mechanism is generic and model-agnostic, which does not rely any particular network structure and can be easily combined with other advanced backbone networks for efficient activity recognition.

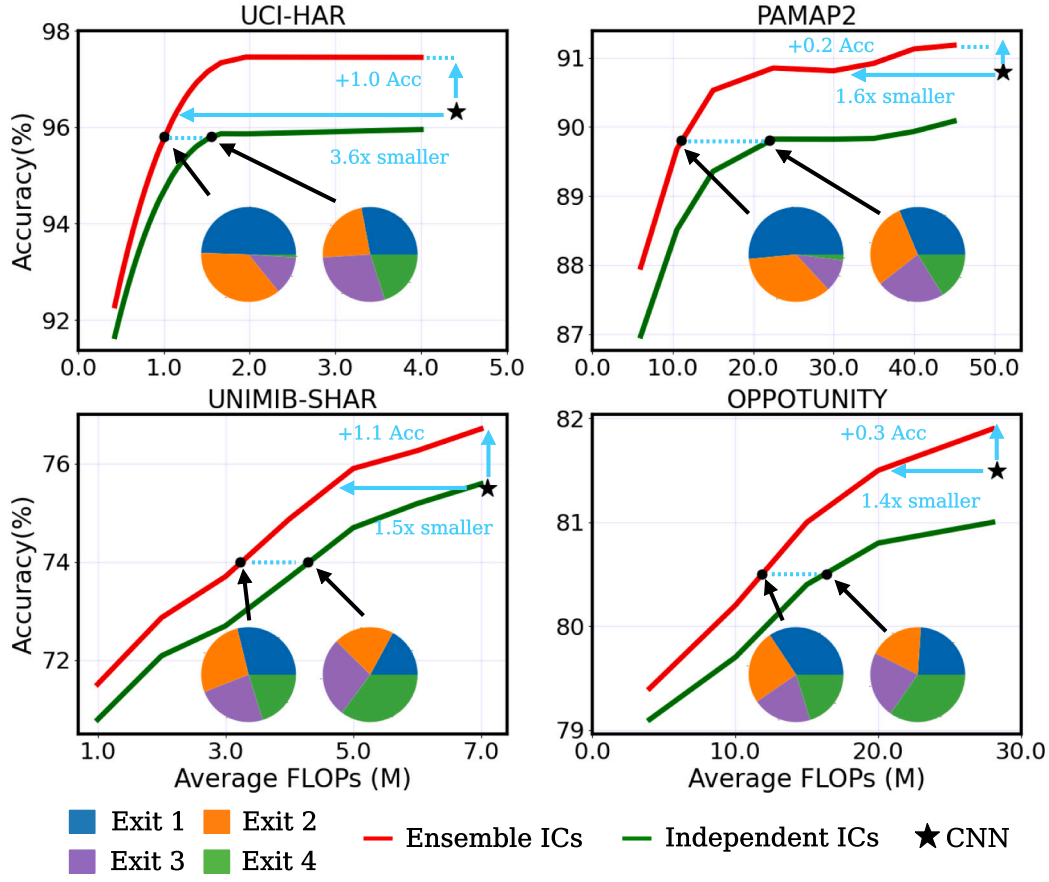


Fig. 4. Budget experiment on ensemble early-exit network and early-exit network (Independent ICs).

4.2.2. Budgeted batch classification and distribution of output

The scenario discussed involves budgeted batch classification, where a budget B of computations and a batch of M instances are provided to a predictive model for classification [35]. To handle this situation, an early-exit mechanism is employed in the network, dynamically evaluating “easy” examples for early termination, while “hard” examples propagate through the entire network.

By controlling the exit threshold of the network, we can control the computational cost (FLOPs) of processing each sample. By varying the thresholds of all exits, we can obtain a series of cost-accuracy relationships and fit them into a curve. Fig. 4 summarizes the results of budgeted batch classification on the four public HAR benchmark datasets, where two lines represent the cost-accuracy relationships of the early-exit network (Independent ICs) and the ensemble early-exit network proposed by us, and a point representing the case of the baseline CNN. It is important to note that, in the early-exit network (Independent ICs), the ensemble strategy is not employed, and each IC operates independently. We observe that the ensemble early-exit network outperforms the early-exit network (Independent ICs) comprehensively, and as the cost increases, the advantage of the ensemble early-exit network further expands. When compared to CNN, ensemble early-exit network has demonstrated advantages, while early-exit network (Independent ICs) have shown a slight decrease in accuracy. In the figure, we also display the sample exit situation at points A, B, and C to simulate low, medium, and high computational cost scenarios. It is worth mentioning that at point A, we have enabled two internal exits, meaning that samples can only exit through either the first or second exit. Similarly, at point B, we have enabled three internal exits, and at point C, we have enabled all four exits. It can be observed that all exits are fully utilized, and in the case of sufficient budget for

computational cost, samples tend to exit at later exits. This is in line with our expectations. In Fig. 5, the specific paths of sample exits in the three scenarios A, B, and C are shown. To analyze the complexity of different activity samples, we also present a confusion matrix in Fig. 6. In the figure, we can see that simple samples like “Lying” tend to appear in the early layers in all scenarios, while the two most confusing difficult samples like “Standing” and “Sitting” are more likely to exit in the later layers. At the same time, it can be found that in the case of a low cost, samples tend to exit in the early layers, while in the case of a high cost, samples rarely exit in the early layers. This indicates that after more computations, samples tend to exit more frequently in the later layers. The experimental results are in line with our expectations: simple samples exit early in the early layers to save computational resources, while hard samples consume more computation to ensure accuracy.

In Fig. 7, we demonstrate the reasoning process of several specific samples in the network. Through computations in the early layers, it can be observed that simple samples meet the exit threshold, exiting the network and avoiding redundant computations. However, complex samples like “Walking” and “Sitting” are easily confused with other actions in the early layers, making it difficult for the early layers to correctly classify them. By adjusting the threshold, they can undergo more computations in the network and ultimately be classified correctly, which aligns with our expected goal.

4.2.3. Confusion analyses

To demonstrate the superiority of our ensemble early-exit method, we compute the confusion matrices on the UCI-HAR dataset by comparing our approach with the baseline early-exit network (without

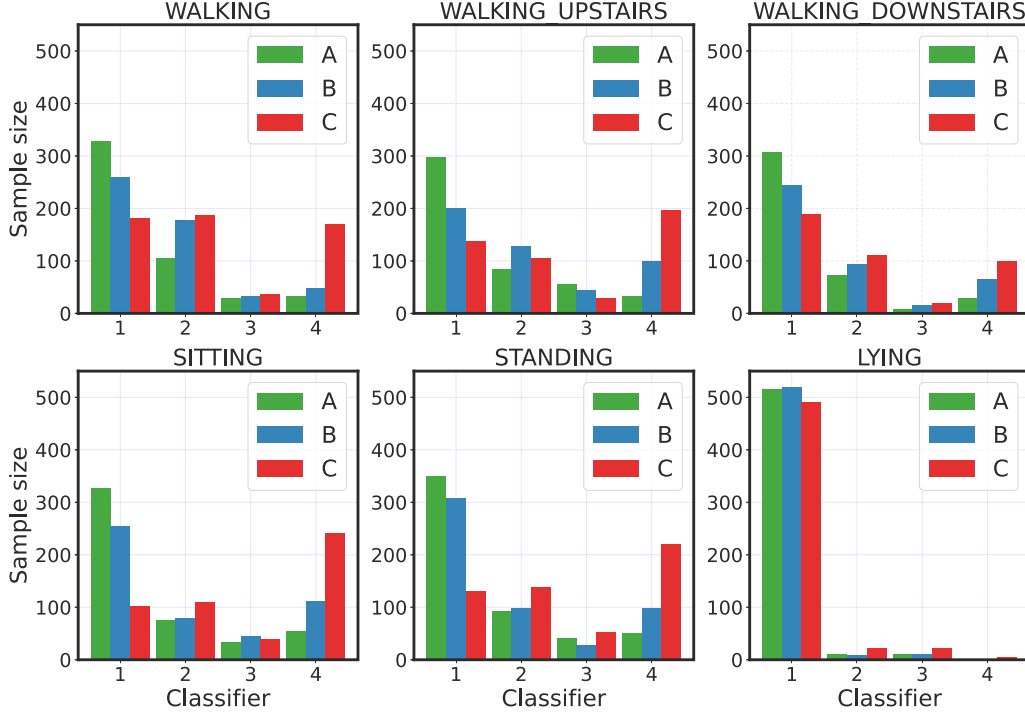


Fig. 5. The output distributions from ICs at different layers. Here A, B, and C refer to three computing scenarios corresponding to low, medium, and high computational budgets, which is controlled based on three confidence thresholds $\tau_1 = \{0.75, 0.75, 0.75\}$, $\tau_2 = \{0.85, 0.85, 0.85\}$ and $\tau_3 = \{0.95, 0.95, 0.95\}$ respectively, as mentioned in Section 4.2.2.

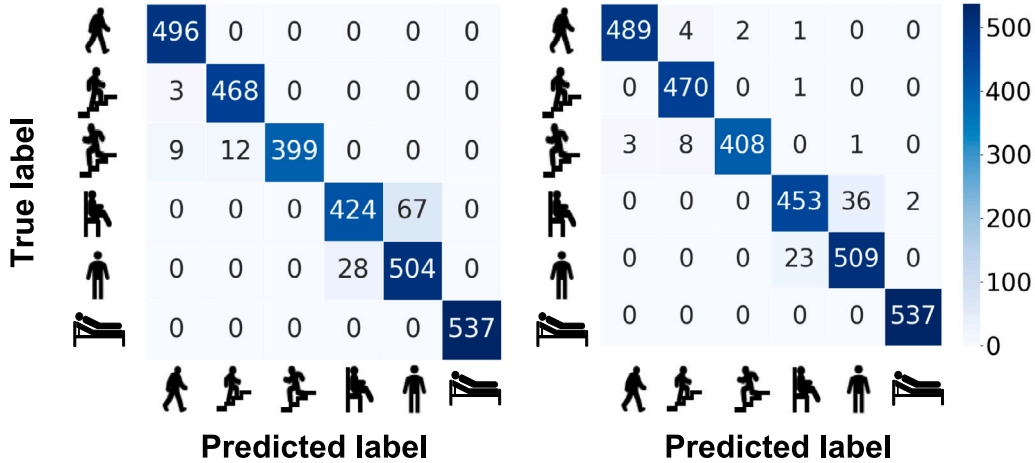


Fig. 6. Confusion matrix of baseline early-exit network (left) and our model (right).

ensemble). As illustrated in Fig. 6, we can observe high confusion between two very similar actions such as “Sitting” and “Standing”, where the baseline early-exit network makes total 95 (i.e., 67+28) errors, while our method results in only 59 (i.e., 36+23) errors. Such confusion may be attributed to their static behavior characteristic. Among “Walking”, “Walking upstairs”, and “Walking downstairs”, there also exists the misclassified action samples due to their somewhat similar signal waveforms. For example, the baseline early-exit network leads to 9 and 12 errors respectively, while misclassifying “Walking downstairs” as “Walking” and “Walking upstairs”. In contrast, our ensemble early-exit network can further decrease the number of misclassified action samples to 3 and 8, respectively. Based on the provided information, we can conclude that the ensemble early-exit network performs better than the baseline early-exit network when it comes to handle complex action samples that are prone to confusion.

5. Overthinking problem on human activity recognition

Adding internal classifiers to the backbone network allows us to study the problem of overthinking on CNNs. Our definition of overthinking with regards to an input sample is the network’s capacity to arrive at an accurate internal prediction. Formally, the network overthinks on the input sample (x, y) if $\hat{y}_i = y$, i.e. the i th internal prediction is a correct one.

Overthinking the problem leads to the network reaching the correct prediction in the early layers, while the remaining forward propagation will lead to a waste of computation. In addition, in some cases, overthinking can lead to eventual misclassification, i.e., $\hat{y}_i = y \neq y_{final}$, where y_{final} indicates the final prediction. Even though the internal prediction was correct (i.e., $\hat{y}_i = y$), the network still may end up outputting the wrong prediction y_{final} that does not match the real

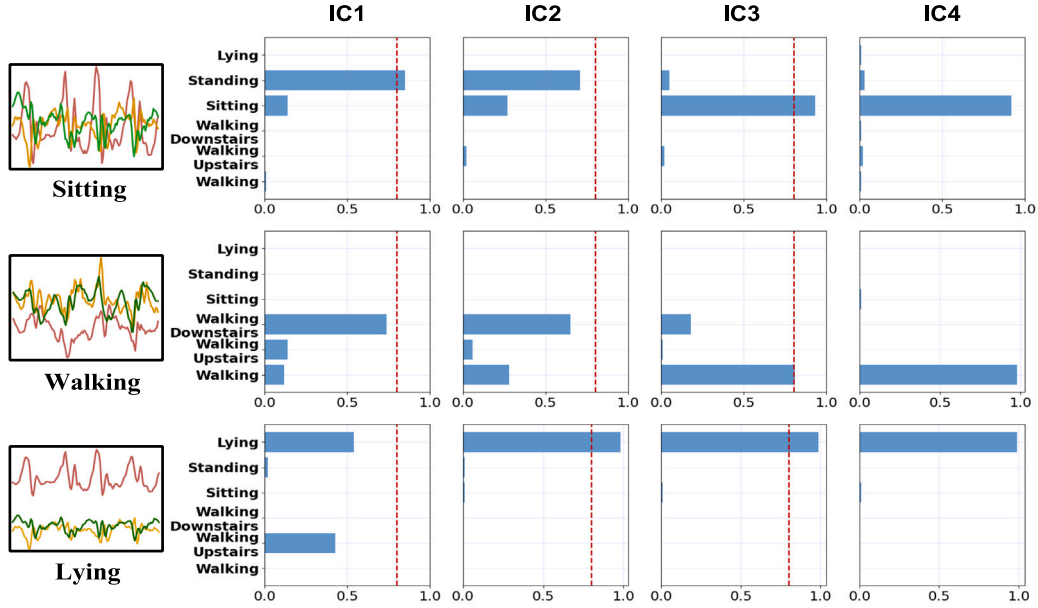


Fig. 7. The thinking process of specific samples in the network, where y-axis indicates total target activity classes and x-axis indicates their predicted softmax probability at different layers.

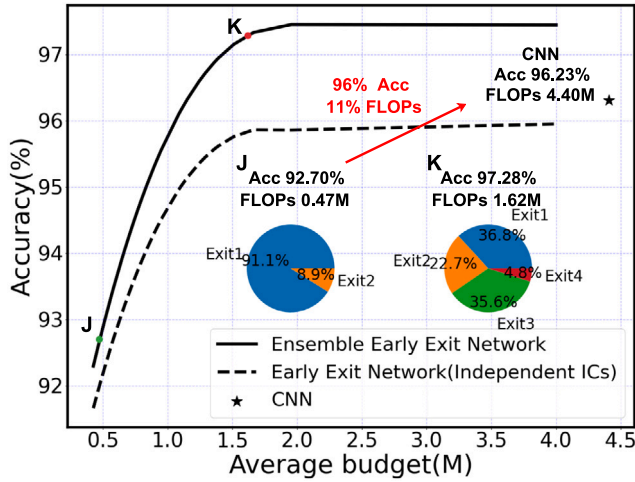


Fig. 8. Waste caused by overthinking problem on UCI-HAR dataset.

label y (i.e., $y \neq y_{final}$). It should be noted that overthinking, unlike overfitting, results in both computational inefficiency and misclassifications. Additionally, while regularization techniques like Dropout can help alleviate overfitting, there is no universal solution for addressing overthinking [15].

To illustrate the overthinking problem, we remove the ensemble layer and conditional prediction from the network in Section 3 and use only the backbone network and the fully connected layer as internal classifiers. Our training method uses separate training of the backbone network and the internal classifier. Through this conversion, we can observe the progression of the initial network's predictions without any modification.

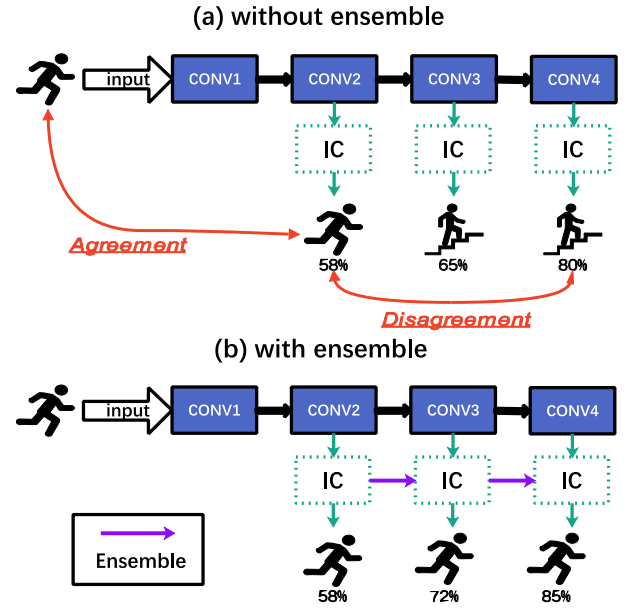


Fig. 9. Destructive effect of overthinking Problem in (a) and mitigating the overthink problem through ensemble in (b).

5.1. Waste caused by overthinking problem

On the HAR datasets, the difficulty level of activity samples varies, with most of the samples being correctly predicted without going through the full network, a phenomenon that is particularly evident on simple datasets such as the UCI-HAR dataset. By using a backbone network plus a fully connected layer as an internal classifier, we can observe the exit of samples. It is worth mentioning that in this

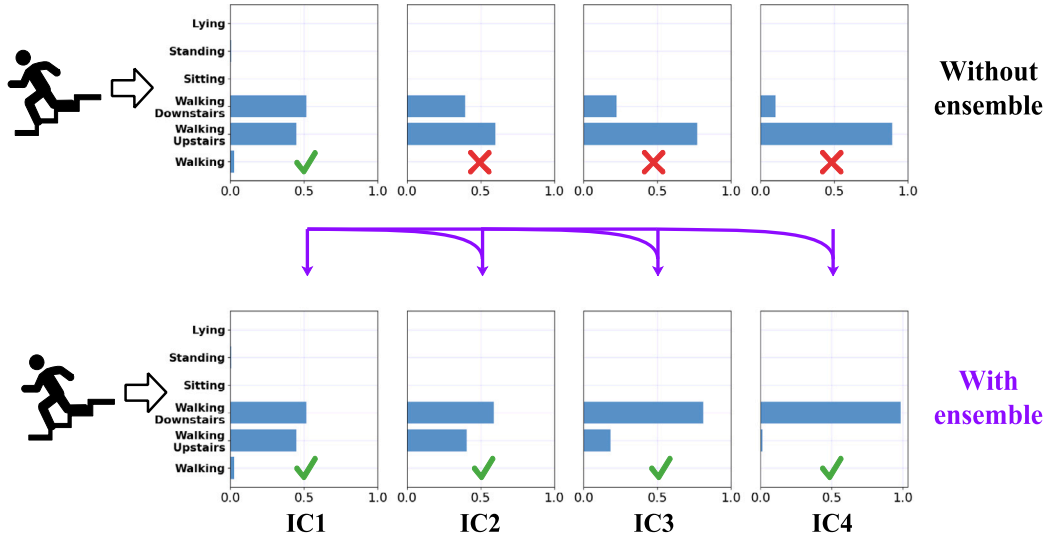


Fig. 10. The thought process of specific sample in the early-exit network without ensemble, which is misclassified due to overthinking problem. Ensemble enables the sample within the network to ultimately generate accurate prediction.

experiment we only focus on the samples that were correctly predicted. From our experiments as illustrated in Fig. 8, we can see that for the UCI-HAR dataset, about 90% of the samples can be correctly classified at the first exit, while only no more than 5% of the samples need to reach the final layer to be correctly classified which means that 90% accuracy of the original network can be achieved using less than 25% computational effort of the original network. The later layer is only used for a very small proportion of complex samples. By using the confidence-based conditional inference, we can make simple samples that do not pass through the entire network to alleviate the waste of computational resources caused by overthinking problem.

5.2. Destructive effect of overthinking problem

In the previous content, we have discussed the computational waste when correctly classifying samples. Next we focus on those samples that are misclassified. As shown in Fig. 9, we observe that there are some samples that can be correctly classified in the early classifier, but are misclassified in the later layers. We categorize this as the destructive effect of overthink. We conjecture that it is the early layers that have the features that the later layers do not have that allow them to make correct predictions. In Fig. 10, we present several observed specific examples where the samples are correctly classified in the early layers but produce errors in the late layers. Theoretically, this problem can be addressed by adjusting the threshold of the internal classifier. However, in practice, changing the threshold also leads to premature termination of other normally classified samples, resulting in errors. To address this issue fundamentally, we propose an ensemble early-exit network (Section 3) that integrates voting to prevent individual exits from making independent judgments, thereby alleviating overthinking. We also apply a metric named Hindsight Improvability (HI) [32] to evaluate how much the IC improves performance by reusing features. We call this metric Hindsight Improvability since it quantifies the number of errors that could have been prevented if past information had been utilized effectively.

Let C_m represent the set of instances correctly classified by IC_m , while its complement set $\bar{C}_m$, refers to the set of instances classified incorrectly. To assess the Hindsight Improvability of IC_m , we compute the following:

$$HI_m = \frac{\bar{C}_m \cap (U_{n < m} C_n)}{\bar{C}_m}, \quad (10)$$

which is used to measure how many mistakes one is able to avoid while combining the past information efficiently.

Fig. 11 compares the HI values on the UCI-HAR dataset for a network using independent ICs and a network with ensemble ICs. We can observe that the HI values for the network using the ensemble ICs are consistently lower than for the baseline, and a greater difference can be seen in the later ICs. Thus, the ensemble ICs are able to effectively use past information to reduce errors caused by overthinking.

5.3. Confusion score

The mid prediction of the early-exit network reveals whether the network's thinking process is consistently in reaching its final prediction. Disagreements among predictions indicate inconsistency and confusion in thinking, while an agreement represent consistency. With the proposed ensemble early-exit network, we can observe this phenomenon and calculate the network's confusion score. The confusion score quantifies the degree of difference between the final prediction and the internal predictions. The confusion score is obtained by calculating the L1 distance between the internal predictions and the mid-training predictions, i.e. $D_i(x) = |q_{final}(x) - q_i(x)|$, in which the i means the index of internal classifier. The sum of all internal predictions, denoted as $\sum_i D_i(x)$, gives the confusion score on sample x .

The confusion scores of correct and incorrect classified samples are shown in Fig. 12. It can be observed that the misclassified samples have higher confusion scores, indicating that their internal thinking process in the network is more inconsistent compared to the correctly classified samples, making them more prone to confusion. The average confusion score for correct samples is 0.95, while for incorrect samples it is 2.52. The difference between them exceeds one standard deviation 1.23. As a baseline, we consider the predicted confidence scores. The difference in confidence scores is less pronounced: 0.94 vs. 0.73 on the correct and wrong predictions, respectively and less than one standard deviation difference. Therefore, we can use the confusion score as an indicator of whether the network will misclassify a sample. Such an indicator is practically meaningful for handling errors: for example, it can alert users to situations where the network is unable to make accurate predictions, i.e., cases with higher confusion scores. We found that using the confusion score as an indicator correctly predicted 58.0% of the misclassified samples, while using confidence scores only predicted 40.1% of them. In conclusion, the confusion score is a reliable and non-expensive indicator of potential misclassification.

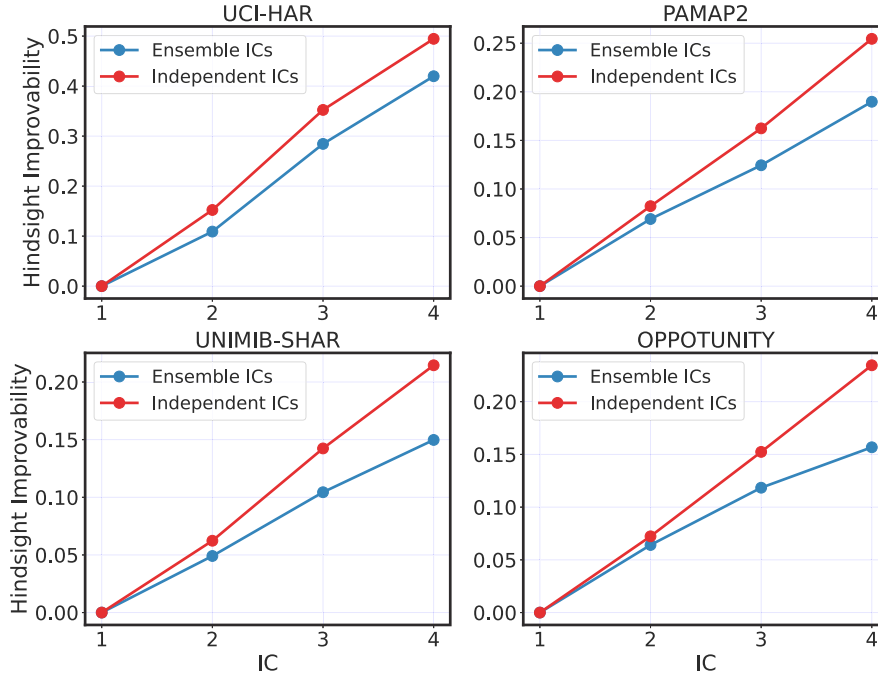


Fig. 11. Hindsight improvability. In respect to each IC on the horizontal axis, we examine the examples that were misclassified and determine the extent to which the preceding ICs classified them correctly. A lower number of such instances reflects a higher proficiency of the IC in reusing previous information.

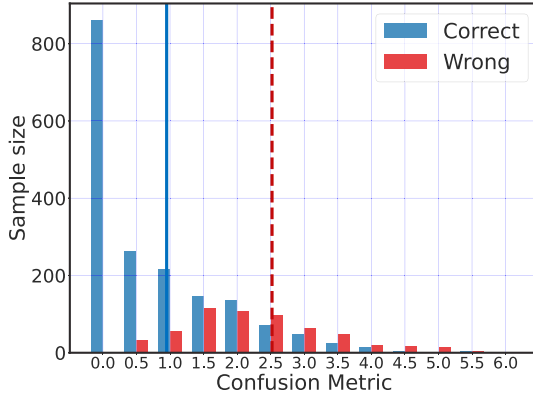


Fig. 12. The distribution of the confusion scores on UNIMIB-SHAR dataset. The straight and dotted lines indicate the average confusion scores in the correct and wrong predictions, respectively.

5.4. Visualization

To investigate the sources of destructive influences in the input, we visualize the confusion in this section. We assume that confusing elements in the input trigger disagreement and misclassification. For a given input (x, y) , we visualize the wrong final prediction and the disagreeing i th correct internal prediction, i.e. $\hat{y}_i = y \neq \hat{y}_{final}$. We use Grad-CAM [54] to visualize the elements of input that are influential in a prediction on PAMAP2 dataset. As we have known, the UCI-HAR dataset is collected through triaxial accelerometer and triaxial gyroscope embedded in a smartphone, whose sensor axes are equal to 6. Such low data dimension is difficult to visualize. To clearly visualize an activity sample along time and sensor axes simultaneously, we choose the PAMAP2 dataset for our analysis, whose data dimension (across sensor axes) is 27 from 3 inertial measurement units. The results are displayed in Fig. 13, where we can see that the first IC focuses on the general features of the input, while the last classifier focuses on the details. This means that the first IC focuses on more features, while

the final classifier is more profound. The confusing visualization helps us to better understand how the network makes wrong decisions, and providing a new perspective on the interpretability of deep learning.

In addition, to demonstrate the necessity and effectiveness of the proposed ensemble early-exit network, we present the process of feature learning and classification in the network. We utilize t-distributed stochastic neighbor embedding (t-SNE) [55] to visualize the features extracted by the ensemble early-exit network after different convolutional layers. Fig. 14 illustrates four 2D sample distribution plots for all test samples, where different colors represent six different motion states. From Figure (a), it can be observed that the features of different motion states in the input are highly mixed and overlapped, making it impossible to distinguish the states. Figure (b) displays the feature discrimination of the early layers, indicating that the early layers have some capability of feature extraction, allowing for slight differentiation between each motion state. Simple categories like “Lying” can be distinguished at the early layers.

Figure (c) demonstrates that after feature extraction in the mid layers, almost all categories can be recognized, although some mixing still exists between the confusable categories “Standing”, “Sitting”, and “Walking upstairs”, “Walking downstairs”, “Walking”. In the late layer visualization, Figure (d), the classification performance is further improved, as “Walking upstairs”, “Walking downstairs” and “Walking” can be well separated, with only a small amount of confusion between “Standing” and “Sitting”. The visualization showcases the varying difficulty levels in classifying different categories, with simple categories being accurately distinguishable at the early layers, while more complex categories require additional layers for precise classification. Therefore, our proposed method achieves satisfactory results in dynamically classifying samples.

6. Actual implementation of Raspberry Pi

To evaluate the real-time performance of our model on mobile devices, we deployed an ensemble early-exit network on an embedded system based on the Raspberry Pi operating system with PyTorch installed. The trained model is imported into the embedded system, and real-time prediction of activities on the UCI dataset is performed.

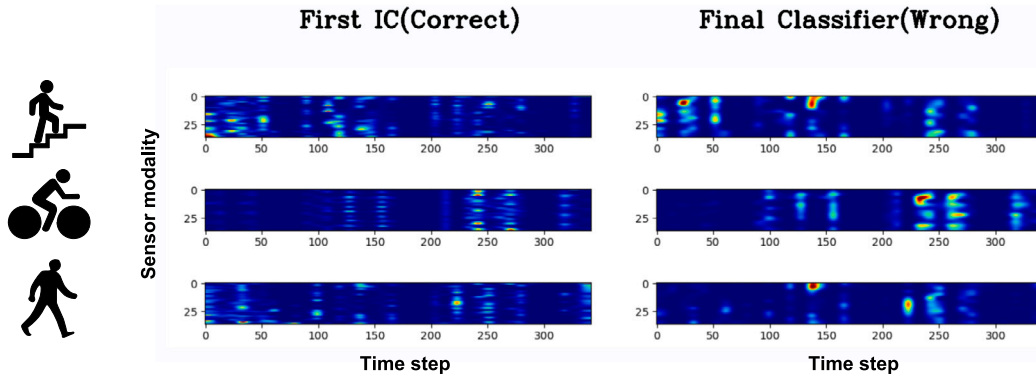


Fig. 13. The visual interpretation of network confusion. The samples are all correctly classified by the first classifier, but incorrectly classified by the last. On the left is the attentional heat map for the first classifier and on the right is the final classifier's.

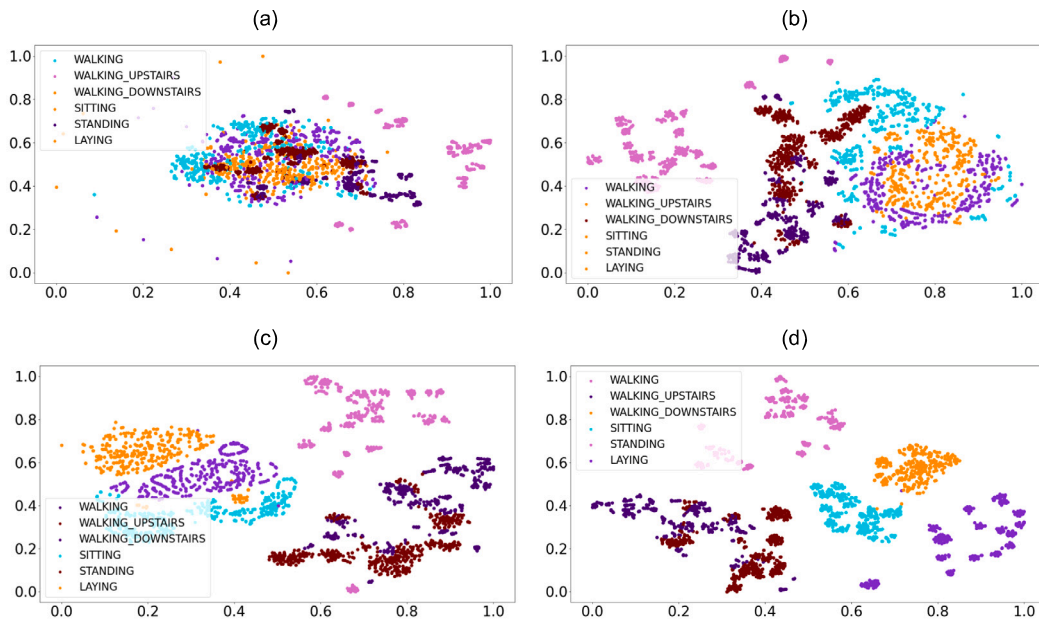


Fig. 14. Feature visualizations of all testing samples from (a) origin input, (b) early layers, (c) mid layers, and (d) late layer. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

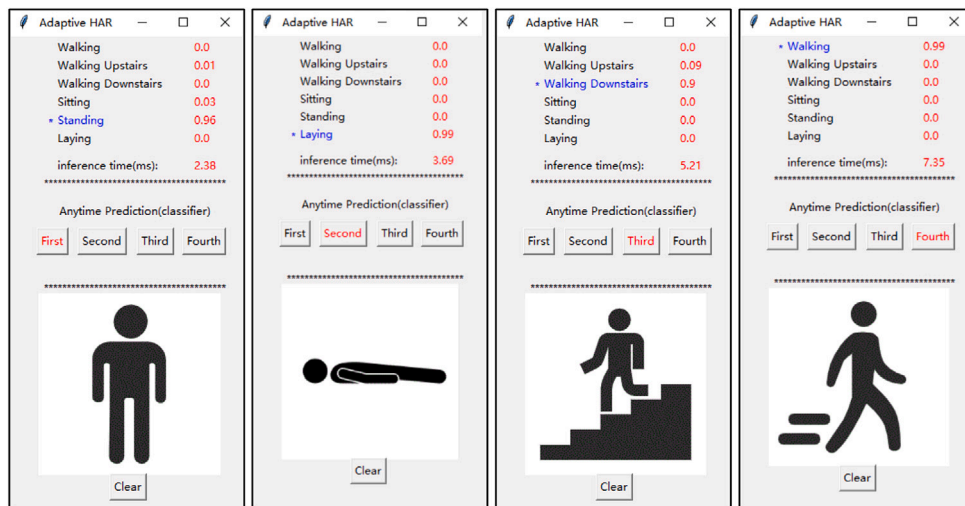


Fig. 15. The user interface of Raspberry Pi: Four pre-set working modes.



Fig. 16. The user interface of Raspberry Pi: Customized exit thresholds for each internal exit.

Table 5

Inference time and accuracy in different modes.

Model	Average time (ms/window)	Test Acc (%)
Baseline	6.89	96.25
First IC	2.35	90.81
Second IC	3.58	94.40
Third IC	5.31	96.30
Final classifier	7.14	97.25

The HAR system is deployed on a Raspberry Pi 4 equipped with the official supported Raspberry Pi operating system, and is configured to communicate with a laptop computer. A Python program is developed for the HAR application. By forcing samples to exit at specific layers, we set up four different modes corresponding to four exits as demonstrated in Fig. 15. In Table 5, we present the average inference time and accuracy of samples under different modes. The results indicate that our model can effectively reduce inference time while maintaining accuracy when deployed on mobile devices. Therefore, the experimental results of this study demonstrate the feasibility of implementing real-time performance of early-exit networks on mobile devices. As shown in Fig. 16, we develop a user interface for customizing three intermediate exit thresholds: low budget {0.75, 0.75, 0.75}, middle budget {0.85, 0.85, 0.85}, and high budget {0.95, 0.95, 0.95}. By setting these confidence thresholds, we can precisely control the average computational cost or inference latency during model inference, which corresponds to the experiment results described in Fig. 17. We consider the batch inference pattern by selecting a total of 500 samples from the UCI-HAR test set. As can be seen in the figure, while our confidence-based inference pipeline reaches a termination threshold, each activity sample that satisfies an early-stopping criterion (i.e., comparing with a pre-defined threshold) will be dynamically exited, demonstrating various inference times based on their different difficult levels. In such a way, a potential user can choose suitable thresholds based on their practically available computational budgets. Following the work using CNN in the literature [28], we measure the energy value for all activity samples using the 1.5 GHz clock speed of Raspberry Pi platform. As shown in Fig. 17, the experimental results validate that the proposed method can effectively accelerating activity inference with a lower energy consumption.

7. Conclusion

Recently, deep neural networks have achieved state-of-the-art performance on various mobile and wearable human activity recognition (HAR) tasks. However, this technology is severely hindered by the current computational power of mobile and wearable devices. The extensive computations in deep learning increase computation time, which is not suitable for real-time HAR on mobile and wearable devices. Shallow and traditional machine learning methods cannot achieve good performance. At the same time, we found that there is a huge difference in the complexity of different samples in HAR tasks, and using the same computation for all samples is wasteful. Therefore, a deep learning method that can balance the trade-off between sample complexity and computation cost while maintaining accuracy is urgently needed.

In this paper, we apply an Ensemble Early-exit Network by cascading multiple internal weaker classifiers into a stronger one, providing an effective solution for HAR on mobile and wearable devices. The proposed Ensemble Early-exit Network is evaluated on four public HAR benchmark datasets, the UCI-HAR dataset, the PAMAP2 dataset, the UNIMIB-SHAR dataset, and the OPPORTUNITY dataset, achieving dynamic routing based on computation cost constraints and sample complexity, while ensuring inference accuracy. We also conduct various experiments to demonstrate the problem of network overthinking in HAR scenarios and its harm, and provide a solution strategy.

Overall, our proposed method can dynamically select the routing of samples in the network based on conditions, thereby reasonably allocating computational resources, which is particularly suitable for HAR tasks that have strict restrictions on computation costs. We hope that this work will inspire future research on early-exit network architecture design in various practical HAR scenarios. In fact, most existing early-exit works typically rely on the confidence-based scores estimated at each intermediate classifier [56], which ignore uncertainty estimates in such early-exit DNNs, thereby making it challenging to build robust HAR systems in real-world scenarios. To mitigate this issue, these intermediate classifiers should not only be accurate, but also know when they are uncertain about current predictions (i.e., when they are likely to be incorrect). In other words, it is crucial that the predicted probabilities are robust, which entails the trustworthy decision-making HAR in early-exit scenario. To the best of our knowledge, the concept of model calibration has been seldom explored for early-exit activity recognition. We plan to tackle such early-exit HAR challenge from a perspective of model calibration, which will be left as future work.

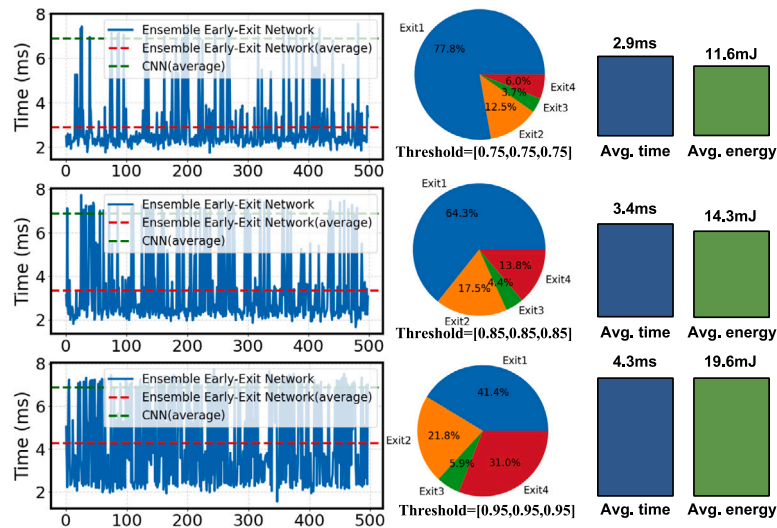


Fig. 17. Inference time statistics under low, middle, and high budgets from top to bottom.

CRedit authorship contribution statement

Jianglai Yu: Software, Methodology, Conceptualization. **Lei Zhang:** Writing – original draft, Data curation. **Dongzhou Cheng:** Visualization. **Can Bu:** Validation. **Liangdong Liu:** Validation. **Hao Wu:** Supervision. **Aiguo Song:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62373194, in part by the Cultivating Plan Program for the Leader in Science and Technology of Yunnan Province, China under Grant 202005AC160005, and in part by the Ten Thousand Talent Plans for Young of Yunnan Province, China under Grant YNWR-QNBJ-2019-188.

Data availability

No data was used for the research described in the article.

References

- [1] L. Minh Dang, Kyungbok Min, Hanxiang Wang, Md. Jalil Piran, Cheol Hee Lee, Hyeonjoon Moon, Sensor-based and vision-based human activity recognition: A comprehensive survey, *Pattern Recognit.* 108 (2020) 107561.
- [2] Yan Wang, Shuang Cang, Hongnian Yu, A survey on wearable sensor modality centred human activity recognition in health care, *Expert Syst. Appl.* 137 (2019) 167–190.
- [3] Andreas Bulling, Ulf Blanke, Bernt Schiele, A tutorial on human activity recognition using body-worn inertial sensors, *ACM Comput. Surv.* 46 (3) (2014) 1–33.
- [4] Chunyu Hu, Yiqiang Chen, Lisha Hu, Xiaohui Peng, A novel random forests based class incremental learning method for activity recognition, *Pattern Recognit.* 78 (2018) 277–290.
- [5] Zhenghua Chen, Chaoyang Jiang, Shili Xiang, Jie Ding, Min Wu, Xiaoli Li, Smartphone sensor-based human activity recognition using feature fusion and maximum full a posteriori, *IEEE Trans. Instrum. Meas.* 69 (7) (2019) 3992–4001.
- [6] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, Jorge L. Reyes-Ortiz, Human activity recognition on smartphones using a multiclass hardware-friendly support vector machine, in: *Ambient Assisted Living and Home Care: 4th International Workshop, IWAAL 2012, Vitoria-Gasteiz, Spain, December 3–5, 2012. Proceedings 4*, Springer, 2012, pp. 216–223.
- [7] Jindong Wang, Yiqiang Chen, Shuji Hao, Xiaohui Peng, Lisha Hu, Deep learning for sensor-based activity recognition: A survey, *Pattern Recognit. Lett.* 119 (2019) 3–11.
- [8] Yu Cheng, Duo Wang, Pan Zhou, Tao Zhang, A survey of model compression and acceleration for deep neural networks, 2017, arXiv preprint arXiv:1710.09282.
- [9] Simone Scardapane, Michele Scarpiniti, Enzo Baccarelli, Aurelio Uncini, Why should we add early exits to neural networks? *Cogn. Comput.* 12 (5) (2020) 954–966.
- [10] Lei Deng, Guoqi Li, Song Han, Luping Shi, Yuan Xie, Model compression and hardware acceleration for neural networks: A comprehensive survey, *Proc. IEEE* 108 (4) (2020) 485–532.
- [11] Yanjiao Chen, Baolin Zheng, Zihan Zhang, Qian Wang, Chao Shen, Qian Zhang, Deep learning on mobile and embedded devices: State-of-the-art, challenges, and future directions, *ACM Comput. Surv.* 53 (4) (2020) 1–37.
- [12] Jiahui Yu, Linjie Yang, Ning Xu, Jianchao Yang, Thomas Huang, Slimmable neural networks, 2018, arXiv preprint arXiv:1812.08928.
- [13] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, Yulin Wang, Dynamic neural networks: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (11) (2021) 7436–7456.
- [14] Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012).
- [15] Yigitcan Kaya, Sanghyun Hong, Tudor Dumitras, Shallow-deep networks: Understanding and mitigating network overthinking, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 3301–3310.
- [16] Surat Teerapittayanon, Bradley McDanel, Hsiang-Tsung Kung, Branchynet: Fast inference via early exiting from deep neural networks, in: *2016 23rd International Conference on Pattern Recognition, ICPR, IEEE, 2016*, pp. 2464–2469.
- [17] Xin Dai, Xiangnan Kong, Tian Guo, EPNet: Learning to exit with flexible multi-branch network, in: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, 2020, pp. 235–244.
- [18] Zehua Sun, Qihong Ke, Hossein Rahmani, Mohammed Bannamoun, Gang Wang, Jun Liu, Human action recognition from various data modalities: A review, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (3) (2022) 3200–3225.
- [19] Henry Friday Nweke, Ying Wah Teh, Mohammed Ali Al-Garadi, Uzoma Rita Alo, Deep learning algorithms for human activity recognition using mobile and wearable sensor networks: State of the art and research challenges, *Expert Syst. Appl.* 105 (2018) 233–261.
- [20] Ming Zeng, Le T. Nguyen, Bo Yu, Ole J. Mengshoel, Jiang Zhu, Pang Wu, Joy Zhang, Convolutional neural networks for human activity recognition using mobile sensors, in: *6th International Conference on Mobile Computing, Applications and Services*, IEEE, 2014, pp. 197–205.
- [21] Andrey Ignatov, Real-time human activity recognition from accelerometer data using convolutional neural networks, *Appl. Soft Comput.* 62 (2018) 915–922.
- [22] Jianbo Yang, Minh Nhut Nguyen, Phyo Phyo San, Xiaoli Li, Shonali Krishnaswamy, Deep convolutional neural networks on multichannel time series for human activity recognition, in: *Ijcai*, Vol. 15, Buenos Aires, Argentina, 2015, pp. 3995–4001.
- [23] Francisco Javier Ordóñez, Daniel Roggen, Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition, *Sensors* 16 (1) (2016) 115.
- [24] Fei Luo, Salabat Khan, Yandao Huang, Kaishun Wu, Binarized neural network for edge intelligence of sensor-based human activity recognition, *IEEE Trans. Mob. Comput.* 22 (3) (2021) 1356–1368.

- [25] Eunji Kim, Interpretable and accurate convolutional neural networks for human activity recognition, *IEEE Trans. Ind. Inform.* 16 (11) (2020) 7190–7198.
- [26] Haojie Ma, Wenzhong Li, Xiao Zhang, Songcheng Gao, Sanglu Lu, AttnSense: Multi-level attention mechanism for multimodal human activity recognition, in: *IJCAI*, 2019, pp. 3109–3115.
- [27] Emanuele Lattanzi, Chiara Contoli, Valerio Freschi, Do we need early exit networks in human activity recognition? *Eng. Appl. Artif. Intell.* 121 (2023) 106035.
- [28] Nafii Rashid, Berken Utku Demirel, Mohammad Abdullah Al Faruque, AHAR: Adaptive CNN for energy-efficient human activity recognition in low-power edge devices, *IEEE Internet Things J.* 9 (15) (2022) 13041–13051.
- [29] Lorena Qendro, Alexander Campbell, Pietro Lio, Cecilia Mascolo, Early exit ensembles for uncertainty quantification, in: *Machine Learning for Health*, PMLR, 2021, pp. 181–195.
- [30] Tianxiang Sun, Yunhua Zhou, Xiangyang Liu, Xinyu Zhang, Hao Jiang, Zhao Cao, Xuanjing Huang, Xipeng Qiu, Early exiting with ensemble internal classifiers, 2021, arXiv preprint arXiv:2105.13792.
- [31] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, Furu Wei, Bert loses patience: Fast and robust inference with early exit, *Adv. Neural Inf. Process. Syst.* 33 (2020) 18330–18341.
- [32] Maciej Wołczyk, Bartosz Wójcik, Klaudia Bałazy, Igor T. Podolak, Jacek Tabor, Marek Śmieja, Tomasz Trzcinski, Zero time waste: Recycling predictions in early exit neural networks, *Adv. Neural Inf. Process. Syst.* 34 (2021) 2516–2528.
- [33] Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, Zhuowen Tu, Deeply-supervised nets, in: *Artificial Intelligence and Statistics*, Pmlr, 2015, pp. 562–570.
- [34] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [35] Gao Huang, Danlu Chen, Tianhong Li, Felix Wu, Laurens van der Maaten, Kilian Weinberger, Multi-scale dense networks for resource efficient image classification, in: *International Conference on Learning Representations*, 2018.
- [36] Chi Li, M. Zeeshan Zia, Quoc-Huy Tran, Xiang Yu, Gregory D. Hager, Manmohan Chandraker, Deep supervision with shape concepts for occlusion-aware 3d object parsing, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5465–5474.
- [37] Jiwon Kim, Jung Kwon Lee, Kyoung Mu Lee, Deeply-recursive convolutional network for image super-resolution, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1637–1645.
- [38] Yishuo Zhang, Albert C.S. Chung, Deep supervision with additional labels for retinal vessel segmentation task, in: *Medical Image Computing and Computer Assisted Intervention–MICCAI 2018: 21st International Conference*, Granada, Spain, September 16–20, 2018, *Proceedings, Part II* 11, Springer, 2018, pp. 83–91.
- [39] Renjie Li, Xinyi Wang, Guan Huang, Wenli Yang, Kaining Zhang, Xiaotong Gu, Son N. Tran, Saurabh Garg, Jane Alty, Quan Bai, A comprehensive review on deep supervision: Theories and applications, 2022, arXiv preprint arXiv:2207.02376.
- [40] Balaji Lakshminarayanan, Alexander Pritzel, Charles Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [41] Stanislav Fort, Huiyi Hu, Balaji Lakshminarayanan, Deep ensembles: A loss landscape perspective, 2019, arXiv preprint arXiv:1912.02757.
- [42] Zanooby N. Khan, Jamil Ahmad, Attention induced multi-head convolutional neural network for human activity recognition, *Appl. Soft Comput.* 110 (2021) 107671.
- [43] Attila Reiss, Didier Stricker, Introducing a new benchmarked dataset for activity monitoring, in: *2012 16th International Symposium on Wearable Computers*, IEEE, 2012, pp. 108–109.
- [44] Ming Zeng, Haoxiang Gao, Tong Yu, Ole J. Mengshoel, Helge Langseth, Ian Lane, Xiaobing Liu, Understanding and improving recurrent networks for human activity recognition by continuous attention, in: *Proceedings of the 2018 ACM International Symposium on Wearable Computers*, 2018, pp. 56–63.
- [45] Ricardo Chavarriaga, Hesam Sagha, Alberto Calatroni, Sundara Tejaswi Digmarti, Gerhard Tröster, José del R. Millán, Daniel Roggen, The Opportunity challenge: A benchmark database for on-body sensor-based activity recognition, *Pattern Recognit. Lett.* 34 (15) (2013) 2033–2042.
- [46] Nils Y. Hammerla, Shane Halloran, Thomas Plötz, Deep, convolutional, and recurrent models for human activity recognition using wearables, 2016, arXiv preprint arXiv:1604.08880.
- [47] Daniela Micucci, Marco Mobilio, Paolo Napoletano, Unimib shar: A dataset for human activity recognition using acceleration data from smartphones, *Appl. Sci.* 7 (10) (2017) 1101.
- [48] Yilin Dong, Xinde Li, Jean Dezert, Rigui Zhou, Changming Zhu, Lai Wei, Shuzhi Sam Ge, Evidential reasoning with hesitant fuzzy belief structures for human activity recognition, *IEEE Trans. Fuzzy Syst.* 29 (12) (2021) 3607–3619.
- [49] Po Yang, Congmin Yang, Vitaveska Lanfranchi, Fabio Ciravegna, Activity graph based convolutional neural network for human activity recognition using acceleration and gyroscope data, *IEEE Trans. Ind. Inform.* 18 (10) (2022) 6619–6630.
- [50] Kaixuan Chen, Lina Yao, Dalin Zhang, Bin Guo, Zhiwen Yu, Multi-agent attentional activity recognition, 2019, arXiv preprint arXiv:1905.08948.
- [51] Frédéric Li, Kimiaki Shirahama, Muhammad Adeel Nisar, Lukas Köping, Marcin Grzegorzec, Comparison of feature learning methods for human activity recognition using wearable sensors, *Sensors* 18 (2) (2018) 679.
- [52] Jun Long, Wuqing Sun, Zhan Yang, Osolo Ian Raymond, Asymmetric residual neural network for accurate human activity recognition, *Information* 10 (6) (2019) 203.
- [53] Sungho Suh, Vitor Fortes Rey, Paul Lukowicz, TASKED: Transformer-based Adversarial learning for human activity recognition using wearable sensors via Self-Knowledge Distillation, *Knowl.-Based Syst.* 260 (2023) 110143.
- [54] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, Dhruv Batra, Grad-cam: Visual explanations from deep networks via gradient-based localization, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 618–626.
- [55] Laurens Van der Maaten, Geoffrey Hinton, Visualizing data using t-SNE, *J. Mach. Learn. Res.* 9 (11) (2008).
- [56] Lassi Meronen, Martin Trapp, Andrea Pilzer, Le Yang, Arno Solin, Fixing overconfidence in dynamic neural networks, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 2680–2690.



Jianglai Yu received the B.S. degree (2022) from Nanjing Normal University, Nanjing, China. He is currently pursuing the M.S. degree with Nanjing Normal University. His research interests include activity recognition, computer vision, and machine learning.



Lei Zhang received the B.Sc. degree (2001) in computer science from Zhengzhou University, China, and the M.S. degree (2004) in pattern recognition and intelligent system from Chinese Academy of Sciences, China. He received the Ph.D. degree (2007) from Southeast University, China, in 2011. He was a Research Fellow (2008) with IPAM, UCLA, Los Angeles. He is currently an Associate Professor with the School of Electrical and Automation Engineering, Nanjing Normal University. His research interests include machine learning, human activity recognition and computer vision.



Dongzhou Cheng is currently pursuing the master's degree with Nanjing Normal University in China. His work has been published in international peer reviewed journals such as *IEEE Transactions on Instrumentation and Measurement*, *IEEE Journal of Biomedical and Health Informatics*, *IEEE Sensors Journal*, *Elsevier Knowledge-Based Systems*, *Elsevier Expert Systems with Applications*. His research interests include Ubiquitous Computing, Federated Learning, and Self-Supervised Learning.



Can Bu received the B.S. degree from Huaiyin Institute of Technology, Huaian, China, in 2021. He is currently pursuing the M.S. degree with Nanjing Normal University. His research interests include activity recognition, federated learning, and machine learning.



Liangdong Liu received the B.S. degree from Nanjing Normal University, Nanjing, China, in 2022. He is currently pursuing the M.S. degree with Nanjing Normal University. His research interests include activity recognition, computer vision, and machine learning.



Hao Wu received the B.S. degree (2001) in computer science from Zhengzhou University, He received M.S. degree (2004) and Ph.D. degree (2007) in computer science from Huazhong University of Science and Technology. Now, he is an associate professor at School of Information Science and Engineering, Yunnan University, China. He has published more than 50 papers in peer-reviewed international journals and conferences, such as JASIST, FGCS, J.Supercomputing, KBS and PUC. He has coauthored a monograph published in World Scientific. He has also served as reviewers and PC members for many venues. His research interests include service computing, information filtering and recommender systems.



Aiguo Song (Senior Member, IEEE) received the Ph.D. degree (1996) in measurement and control from Southeast University, Nanjing. He is currently a Professor with the School of Instrument Science and Engineering, Southeast University. He is the chair of the China Chapter of the IEEE Robotics and Automation Society. His current research interests include teleoperation, haptic display, the Internet telerobotics, distributed measurement systems and machine learning.